# ТЗ: SputnikOffer Bot — мини-CRM для менторства в Telegram

> **Версия:** 1.0 (MVP) **Формат:** Telegram-бот (всё в боте, без отдельной веб-админки) **Язык интерфейса:** Русский

---

## 1. Назначение

Telegram-бот, выполняющий роль мини-CRM для одного ментора и его учеников. Поддерживает направления (Rust / Python / C++), этапы прохождения внутри направления, weekly-отчёты учеников, дедлайны и аналитику для ментора.

Кабинет ментора и кабинет ученика реализованы как меню внутри одного бота, доступ разграничивается по `telegram_user_id`.

---

## 2. Роли

| Роль | Идентификация | Доступ |
|---|---|---|
| Ментор | `telegram_user_id` присутствует в таблице `mentors` | Полное меню ментора: выпуск кодов, ученики, управление этапами, экспорт |
| Ученик | `telegram_user_id` присутствует в таблице `students` | Меню ученика: прогресс, дедлайны, weekly |
| Гость | Не зарегистрирован | Только `/start` → приглашение ввести код доступа |

Первый ментор создаётся миграцией / SQL-скриптом при первом разворачивании (значения берутся из env: `BOOTSTRAP_MENTOR_TG_ID`, `BOOTSTRAP_MENTOR_USERNAME`).

---

## 3. Технологический стек

- **Язык / фреймворк:** Python 3.12+, aiogram 3.x (FSM «из коробки», async).
- **БД:** PostgreSQL 15+ (схема ниже).
- **ORM / миграции:** SQLAlchemy 2.x + Alembic.
- **Планировщик:** APScheduler (cron-задачи для уведомлений и weekly-кнопки).
- **Excel-экспорт:** `openpyxl`.
- **Деплой:** Docker + docker-compose, single instance (нагрузка низкая).
- **Часовой пояс:** всё в БД хранится в UTC. У каждого ученика своя TZ в поле `students.timezone` (IANA-формат, например `Europe/Moscow`, `Asia/Yekaterinburg`,

`Asia/Singapore`). Расчёт «текущей недели», времени weekly-ремайндеров и дедлайнов — в TZ конкретного ученика. Дефолтная TZ для нового ученика берётся из env `DEFAULT_STUDENT_TZ` (рекомендую `Europe/Moscow`).

- **Логирование:** структурированные логи (`structlog` или JSON через `logging`), уровень `INFO`.
- **UX-принцип:** вся навигация — через кнопки (reply-клавиатура для главного меню + inline-клавиатуры для контекстных действий), а не команды. Единственная поддерживаемая slash-команда — `/start` (служебная, требуется Telegram). Никаких `/mentor`, `/me`, `/help` и т.п. — те же действия достигаются нажатием кнопки.

---

## 4. Модель данных

Существующая схема (`directions`, `students`, `stages`, `student_stage_progress`, `daily_progress`, `student_tasks` + ENUMs) принимается как основа. Ниже — необходимые дополнения и уточнения.

### 4.1 Новые таблицы

**`mentors`**

```sql
CREATE TABLE mentors (
  id               bigserial PRIMARY KEY,
  telegram_user_id bigint UNIQUE NOT NULL,
  telegram_username text,
  display_name     text NOT NULL,
  is_active        boolean NOT NULL DEFAULT true,
  created_at       timestamptz NOT NULL DEFAULT now()
);
```

**`access_codes`**

Одноразовые коды для регистрации учеников. Код живёт до первого использования (без срока истечения).

```sql
```

```
CREATE TABLE access_codes (
  id            bigserial PRIMARY KEY,
  code          text UNIQUE NOT NULL,             -- 8 символов, A-Z0-9 (без 0/O/1/I
  direction_id  smallint NOT NULL REFERENCES directions(id),
  created_by    bigint NOT NULL REFERENCES mentors(id),
  used_by       bigint REFERENCES students(id),   -- NULL пока не использован
  used_at       timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now(),

  CHECK ((used_by IS NULL AND used_at IS NULL) OR (used_by IS NOT NULL AND used_at I
);

CREATE INDEX idx_access_codes_unused ON access_codes(code) WHERE used_by IS NULL;
```

`notifications_log` **(для дедупликации уведомлений)**

```sql
CREATE TABLE notifications_log (
  id                bigserial PRIMARY KEY,
  recipient_user_id bigint NOT NULL,             -- telegram_user_id
  kind              text NOT NULL,               -- см. §6, типы уведомлений
  related_entity    text,                        -- "stage_progress:123" или "weekl
  sent_at           timestamptz NOT NULL DEFAULT now(),
  UNIQUE (recipient_user_id, kind, related_entity)
);
```

### 4.2 Изменения в существующих таблицах

`daily_progress` **— раздробить** `note` **на структурированные поля**

```sql
ALTER TABLE daily_progress
  ADD COLUMN what_done            text,
  ADD COLUMN problems_solved      text,
  ADD COLUMN problems_help_needed text;
-- старое поле note оставляем как nullable (для совместимости / свободного коммента
```

Семантика полей соответствует исходному сценарию weekly (см. §5.2).

**Расширить** `student_status`

```sql
ALTER TYPE student_status ADD VALUE 'completed';
```

`completed` = ученик получил оффер (см. §5.3.8).

**Добавить часовой пояс ученика в `students`**

```sql
ALTER TABLE students
  ADD COLUMN timezone text NOT NULL DEFAULT 'Europe/Moscow';
-- значение — IANA-имя зоны (Europe/Moscow, Asia/Yekaterinburg, Asia/Singapore и т.д
-- спрашивается при регистрации (см. §5.1), может быть изменено учеником позже (см.
```

**Добавить `is_final` в `stages` (опционально, для удобства)**

```sql
ALTER TABLE stages ADD COLUMN is_final boolean NOT NULL DEFAULT false;
-- ограничение: not more than one is_final=true per direction
CREATE UNIQUE INDEX uq_one_final_per_direction
  ON stages(direction_id) WHERE is_final = true;
```

Альтернатива — определять финальный этап как `MAX(stage_number)` внутри direction; решение на усмотрение разработчика.

### 4.3 Замечания по существующим полям

- `students.target_market` — заполняется при регистрации/в процессе, желаемое место работы (опционально).
- `students.offer` — заполняется при завершении (название компании, позиция).
- `student_stage_progress.mentor_comment` — в MVP **не используется** (см. §8).
- `student_tasks` — таблица в схеме есть, но функциональность **в скоуп MVP не входит** (см. §8).
- `daily_progress.status` — для MVP всегда `active`. Значения `no_response`/`paused`/`dropped` зарезервированы под будущее (если решим хранить пропуски как записи).

---

## 5. Функциональные сценарии

### 5.1 Регистрация ученика

**Создание кода (ментор)**

В кабинете ментора → «➕ Новый код доступа»:

1. Бот спрашивает направление (inline-кнопки из `directions`, где `is_active`).

2. Бот генерирует 8-символьный код (`[A-Z0-9]`, исключая `0/O/1/I`), сохраняет в `access_codes` без срока истечения (код живёт до первого использования).
3. Бот возвращает ментору сообщение с кодом + deep-link: `https://t.me/<bot_username>?start=<code>`.

**Активация кода (ученик)**

1. Ученик переходит по deep-link или присылает `/start`.
2. Если у `telegram_user_id` уже есть запись в `students` → «Вы уже зарегистрированы», показать главное меню ученика.
3. Иначе — бот просит ввести код (или код приходит из deep-link автоматически).
4. Валидация кода:
   - Не найден → «Код не найден».
   - `used_by IS NOT NULL` → «Код уже использован».
5. Бот спрашивает часовой пояс ученика (inline-кнопки):
   - Москва (UTC+3)
   - Калининград (UTC+2)
   - Самара (UTC+4)
   - Екатеринбург (UTC+5)
   - Омск (UTC+6)
   - Красноярск / Новосибирск (UTC+7)
   - Иркутск (UTC+8)
   - Владивосток (UTC+10)
   - Ереван / Тбилиси / Дубай (UTC+4)
   - Алматы / Бишкек (UTC+5/+6)
   - Другой (UTC+N) → выбор смещения от UTC из списка Соответствие «город → IANA-имя» хранится в коде; сохраняем именно IANA-имя в `students.timezone`. Кнопка «Другой» при отсутствии города в списке маппит выбранное смещение на ближайшую подходящую IANA-зону без DST (например, `Etc/GMT-3` для UTC+3).
6. При успехе (всё в одной транзакции):
   - INSERT в `students` (`direction_id` из кода, `telegram_user_id`, `telegram_username`, `name` — из Telegram-профиля, `timezone` — из шага 5, `status = active`, `started_at = today`).
   - INSERT в `student_stage_progress` для **первого** этапа этого направления (минимальный `stage_number`, `is_active=true`): `status='active'`, `started_at=today`, `planned_deadline = today + stages.planned_duration_days`.
   - UPDATE `access_codes`: `used_by`, `used_at = now()`.
   - Уведомление WELCOME: приветственное сообщение, привязанное к направлению (тексты в конфиге / отдельной таблице `welcome_messages` либо в JSON-файле).

Содержит краткий гайд: что такое weekly, где смотреть прогресс.

### 5.2 Отправка weekly (ученик)

#### Условия доступности кнопки «Отправить weekly»

- `student.status = active`.
- За текущую календарную неделю (понедельник — воскресенье в **TZ ученика**) **ещё нет** записи в `daily_progress` с `progress_date = текущий_понедельник_в_TZ_ученика` для этого `student_id`.
- День недели любой (с пн 00:00 по вс 23:59 в TZ ученика).

Если условия не выполнены — кнопка скрыта/задизейблена с пояснением («Уже сдано на этой неделе» / «Аккаунт на паузе»).

#### Поток (FSM, 3 шага)

| Шаг | Текст бота | Тип ввода | Поле в `daily_progress` |
|---|---|---|---|
| 0 | Контекст: «Ты на этапе **{title}** (направление {direction}). Заполним weekly за неделю N.» | — | (только показ) |
| 1 | «Что делал на прошлой неделе? Что нового узнал? Что произошло?» | Свободный текст | `what_done` |
| 2 | «С какими проблемами столкнулся и решил?» (можно пропустить кнопкой) | Свободный текст | `problems_solved` |
| 3 | «С какими проблемами столкнулся и не можешь решить? Нужна помощь?» (можно пропустить кнопкой) | Свободный текст | `problems_help_needed` |

В конце:

1. INSERT в `daily_progress`: `student_id`, `progress_date = текущий_понедельник`, `stage_id = текущий этап`, `status='active'`, `source='bot'`, + три текстовых поля.
2. Сообщение ученику: «Weekly принят, спасибо!»

#### Edge cases

- Юзер начал заполнять, но не закончил → драфт **не сохраняем**. Через 30 минут бездействия — FSM сбрасывается, ученик начнёт заново.
- Двойной быстрый клик «Отправить» → защищаемся UNIQUE-ограничением `(student_id, progress_date)` + транзакцией.

### 5.3 Кабинет ментора

Точка входа: после `/start` бот определяет, что `telegram_user_id` есть в `mentors`, и показывает reply-клавиатуру с главным меню ментора. Клавиатура отображается постоянно — после завершения любого сценария бот возвращает в неё.

Главное меню ментора (reply-кнопки):

1. 👥 **Ученики** — поиск и список
2. 📊 **Общий прогресс** — экспорт в Excel
3. ➕ **Новый код доступа** — см. §5.1
4. 🎯 **Направления и этапы** — CRUD
5. ⚙️ **Дефолтные сроки** — редактирование `planned_duration_days` этапов

#### 5.3.1 Ученики — поиск

Бот спрашивает `@username` / `username` / числовой `telegram_user_id`. Поиск:

```sql
SELECT * FROM students
WHERE telegram_user_id = $1::bigint
   OR lower(telegram_username) = lower($1);
```

- **0 результатов** → «Ученик не найден. Проверьте написание или попросите ученика прислать свой ID.»
- **1 результат** → меню действий по ученику (§5.3.2).
- **>1 результат** (теоретически возможно, если username сменил владельца) → показать inline-список «{display_name} — {direction} — зарегистрирован {date}», выбор пользователя.

#### 5.3.2 Меню действий по ученику

Карточка ученика + кнопки:

- 📈 **Прогресс** — детальный (§5.3.3)
- ➡️ **Перевести на следующий этап** (§5.3.4)
- 🎯 **Выбрать этап вручную** (§5.3.4)
- ⏰ **Изменить дедлайн текущего этапа** (§5.3.5)
- 📅 **Установить дедлайны для всех этапов** (§5.3.5)
- ⏸ **Поставить на паузу** / ▶️ **Возобновить**
- 🚫 **Отчислить** (§5.3.6)
- 🎉 **Завершить — получил оффер** (§5.3.8)

#### 5.3.3 Прогресс ученика

Текстовая карточка:

```
👤 {name} (@{username})
🎯 Направление: {direction}
🟢 Статус: {status}
📅 С нами с: {started_at}

🔷 Текущий этап: {stage_title}
   Старт: {started_at}
   Дедлайн: {planned_deadline}  ⚠️ просрочен на N дней (если есть)

✅ Пройденные этапы:
  1. {title} ({started_at} → {completed_at})
  2. ...

📝 Последние 3 weekly:
  • {date}: {what_done|truncate(120)}
  • ...
```

### 5.3.4 Перевод по этапам

**«Перевести на следующий»**:

1. Бот находит следующий этап (`stage_number = current + 1` у того же направления, `is_active = true`).
2. Подтверждение: «Завершить **{current}** и начать **{next}**?»
3. При подтверждении в одной транзакции:
   - `current_progress.status = 'done'`, `completed_at = today`.
   - INSERT `student_stage_progress` для next: `status='active'`, `started_at=today`, `planned_deadline = today + next.planned_duration_days`.
   - Уведомление STAGE_MOVED ученику.
   - Если `next.is_final = true` → дополнительное сообщение ментору: «{student} теперь на финальном этапе. Когда будет оффер — нажмите 🎉 Завершить.»

**«Выбрать этап вручную»**: список всех этапов направления, при выборе:

- Если уже есть запись `student_stage_progress` для этого этапа — UPDATE `status='active'`, активируем; предыдущий активный → `status='skipped'` или `'done'` (на усмотрение ментора, кнопка выбора).
- Если записи нет — INSERT.

**Валидация**: запрещено перемещать на этап другого направления.

### 5.3.5 Установка дедлайнов

**Изменить дедлайн текущего этапа:**

1. Бот показывает текущий `planned_deadline`.
2. Предлагает варианты: «+7 дней / +14 дней / Своя дата».
3. UPDATE `student_stage_progress.planned_deadline`.
4. Уведомление NEW_DEADLINES ученику.

**Установить дедлайны для всех этапов сразу (дефолт):**

1. Для каждого этапа направления, начиная с текущего, рассчитать `planned_deadline = previous_deadline + planned_duration_days` (для текущего — от `started_at` или от сегодня).
2. Показать ментору список «Этап X: дата Y». Кнопки: «Подтвердить» / «Изменить значение этапа N».
3. При подтверждении — INSERT/UPDATE `student_stage_progress` для всех этапов вперёд (запись со `status='not_started'` если этап ещё не начат).
4. Уведомление NEW_DEADLINES ученику с полным списком.

### 5.3.6 Отчисление ученика

1. Подтверждение «Вы уверены? Это действие можно откатить вручную через БД».
2. UPDATE `students`: `status='dropped'`, `dropped_at=today`.
3. Активные `student_stage_progress` оставляем как есть (история).
4. Уведомление STATUS_CHANGED ученику (опционально, по флагу): «Ваше участие в программе завершено».
5. Weekly-кнопка для ученика более недоступна.

### 5.3.7 Управление направлениями и этапами

**Направления:**

- Список: показать все, активные/неактивные.
- Добавить: ввести `code` (уникальный, например `RUST`), `name` («Rust»).
- Деактивировать (`is_active` нужно добавить в `directions`, см. ниже).
- Удаление — запрещено, если есть привязанные `students` или `stages`.

> **Доработка схемы:** добавить `directions.is_active boolean NOT NULL DEFAULT true`.

**Этапы (внутри направления):**

- Список этапов направления, упорядочен по `stage_number`.
- Добавить этап: `stage_number` (или авто-инкрементом «в конец»), `title`, `description`, `planned_duration_days`, `is_final`.
- Редактировать: те же поля.

- Деактивировать (`is_active=false`).

**Edge cases:**

- Запрещено деактивировать этап, на котором есть `student_stage_progress.status='active'`.
- Изменение `planned_duration_days` не пересчитывает дедлайны уже идущих учеников (дедлайны хранятся в `student_stage_progress.planned_deadline`).
- Изменение `stage_number` — запретить или потребовать пересортировки всей цепочки (проще — запретить, разработчик создаёт новые этапы).

### 5.3.8 Завершение (получил оффер)

Триггер: кнопка 🎉 **Завершить** в меню ученика (доступна, если `student.status = active`).

1. Бот запрашивает: «Компания и позиция?» (свободный текст, сохраняется в `students.offer`).
2. Подтверждение.
3. UPDATE `students`: `status='completed'`, `offer=...`. (Можно дополнительно добавить поле `completed_at date` в `students` — рекомендую.)
4. Если есть активный `student_stage_progress` — `status='done'`, `completed_at=today`.
5. Уведомление STUDENT_OFFER в общий чат учеников (ID чата — `STUDENTS_CHAT_ID` из env):
   > 🎉 **{name}** получил оффер в **{offer}**! Поздравляем!
6. Уведомление лично ученику.

### 5.3.9 Общий прогресс — экспорт Excel

Кнопка «📊 Общий прогресс» генерирует `.xlsx` и присылает файлом.

Листы:

1. **Сводка**
   - Таблица: строки — направления, столбцы — этапы, в ячейках — количество учеников со `status='active'` на этом этапе.
   - Отдельная колонка «На финальном этапе» и «Получили оффер» (всё время).
   - Таблица «Средний срок прохождения этапа» (по завершённым `student_stage_progress`, `AVG(completed_at - started_at)`).
2. **Rust** / **Python** / **C++** (по одному листу на направление)
   - Колонки: имя, @username, текущий этап, дата старта этапа, дедлайн, просрочка (дней), статус.
3. **Дедлайны**
   - Все активные `student_stage_progress` со всех направлений: ученик, направление, этап, дедлайн, просрочка.

- Просроченные строки выделены красным (условное форматирование `openpyxl`).

> Имя файла: `progress_YYYY-MM-DD.xlsx`.

### 5.4 Кабинет ученика

Точка входа: после `/start` бот определяет, что `telegram_user_id` есть в `students`, и показывает reply-клавиатуру с главным меню ученика. Клавиатура отображается постоянно. Кнопка «📝 Отправить weekly» появляется/исчезает в зависимости от условий §5.2.

Главное меню ученика (reply-кнопки):

1. 📈 **Мой прогресс**
2. 📅 **Мои дедлайны**
3. 📝 **Отправить weekly** (см. §5.2; видна только когда доступна)
4. 🌍 **Сменить часовой пояс** — выбор из того же списка, что при регистрации (§5.1, шаг 5); UPDATE `students.timezone`
5. ℹ️ **Помощь**

#### 5.4.1 Мой прогресс

Аналог карточки из §5.3.3, но без блока «Последние weekly» и без действий ментора.

```
🎯 Направление: {direction}
🔹 Текущий этап: {title}
   Старт: {started_at}
   Дедлайн: {planned_deadline}  🟢/🟡/🔴

✅ Пройдено этапов: N
📝 Weekly за всё время: N
```

Цвет дедлайна:

- 🟢 более 7 дней до дедлайна
- 🟡 менее 7 дней
- 🔴 просрочен

#### 5.4.2 Мои дедлайны

Список всех этапов направления с пометками: ✅ пройдено, 🔹 текущий, ⬜ предстоящее.

---

# 6. Уведомления

## 6.1 Типы

| `kind` | Кому | Когда | Текст (шаблон) |
|---|---|---|---|
| `WELCOME` | Ученик | После регистрации | Приветствие + краткий гайд (зависит от направления) |
| `STAGE_MOVED` | Ученик | После перевода на новый этап | «Вас перевели на этап {title}. Дедлайн: {planned_deadline}» |
| `NEW_DEADLINES` | Ученик | После установки/изменения дедлайнов | Список новых дедлайнов |
| `STATUS_CHANGED` | Ученик | После отчисления / возобновления | Соответствующий текст |
| `WEEKLY_REMINDER` | Ученик | Не сдал weekly: вт 18:00, чт 18:00, сб 18:00 (TZ конфига) | «Не забудьте сдать weekly — кнопка в меню» |
| `DEADLINE_APPROACHING` | Ученик | За 3 дня до `planned_deadline`, в день дедлайна | «До дедлайна по этапу {title} осталось N дней» |
| `DEADLINE_OVERDUE` | Ученик | Через 1 день после `planned_deadline` (один раз) | «Дедлайн по этапу {title} просрочен. Свяжитесь с ментором.» |
| `STUDENT_NO_RESPONSE` | Ментор | После 3-го `WEEKLY_REMINDER` подряд без сдачи | «{student} не сдаёт weekly уже {N} недель» |
| `STUDENT_OFFER` | Чат | При завершении ученика (§5.3.8) | «🎉 {student} получил оффер в {company}!» |

## 6.2 Расписание и планировщик

Все cron-задачи запускаются по UTC, но при отборе получателей и форматировании дат используют `students.timezone`.

- **Раз в час** — задача проверяет:
  - `DEADLINE_APPROACHING` / `DEADLINE_OVERDUE`: для всех `student_stage_progress` со `status='active'` — рассчитывается локальная дата ученика (`now_utc → at_timezone(student.tz)::date`) и сравнивается с `planned_deadline`. Отправлять имеет смысл один раз в сутки на нужный offset, поэтому в `notifications_log` запись (см. 6.3) гарантирует, что повторно за тот же offset не отправим.

- `WEEKLY_REMINDER`: для каждого активного ученика берём его локальное время. Если оно сейчас в окне 18:00–18:59 локального времени **и** локальный день недели — `вторник | четверг | суббота` **и** за текущую локальную неделю не было записи в `daily_progress` — отправляем (запись в `notifications_log` предотвращает дубль).

- **Раз в сутки (например, в 00:10 UTC)** — задача пересчитывает кэш «текущий локальный понедельник» для каждого ученика, если в коде используется такой кэш. (Опционально, для оптимизации.)

> Альтернатива: для weekly-ремайндеров и дедлайн-уведомлений можно собирать отдельный per-student cron в APScheduler с триггерами по TZ ученика. Более чисто, но дороже по управлению (при изменении `students.timezone` нужно пересоздавать job). На MVP проще выбрать «час-по-часу с фильтрацией».

### 6.3 Дедупликация

Перед каждой отправкой — проверка `notifications_log` по `(recipient_user_id, kind, related_entity)`. Если запись есть — пропустить. Иначе — отправить и записать.

`related_entity`:

- для `WEEKLY_REMINDER`: `weekly:{ISO_year}-W{ISO_week}:{day_of_week}` (1 запись на каждый день рассылки на каждую неделю);
- для `DEADLINE_APPROACHING` / `DEADLINE_OVERDUE`: `stage_progress:{id}:{offset_label}` (`offset_label` ∈ `before_3d`, `on_day`, `after_1d`);
- для `STAGE_MOVED`, `NEW_DEADLINES`: `stage_progress:{id}:{timestamp}`.

### 6.4 Обработка недоставки

Если Telegram API возвращает 403 (бот заблокирован пользователем):

- Записать в лог (`logger.warning`).
- Не падать, не повторять.
- **Опционально (за рамками MVP):** добавить поле `students.bot_blocked boolean` и отображать в кабинете ментора.

---

## 7. Edge cases и общие требования

- **Сменился `telegram_username`:** при каждом взаимодействии ученика с ботом — UPDATE `students.telegram_username` по `telegram_user_id`.
- **У ученика нет `username`:** разрешено, `CHECK` в схеме это допускает; поиск только по ID.
- **Транзакционность:** все операции, затрагивающие несколько таблиц (регистрация, перевод этапа, отчисление, оффер) — в одной БД-транзакции.

- **Параллельные нажатия:** защищаемся UNIQUE-ограничениями и идемпотентностью обработчиков (повторное нажатие «Завершить» уже-завершённого этапа → молча игнорировать с уведомлением «уже выполнено»).
- **Часовой пояс:** хранится per-student в `students.timezone` (IANA, см. §4.2 и §5.1). Все БД-значения timestamptz — в UTC. При расчёте «текущей недели», локальной даты дедлайна и времени отправки уведомлений — переводим в `students.timezone` функцией `(timestamp AT TIME ZONE 'UTC') AT TIME ZONE student.timezone` (PostgreSQL) или средствами `zoneinfo` в Python. Дефолтная TZ для нового ученика — из env `DEFAULT_STUDENT_TZ` (рекомендую `Europe/Moscow`). Ученик может сменить TZ через свой кабинет (§5.4).
- **Смена TZ ученика:** дедлайны (даты, не времена) и `progress_date` не пересчитываются. Меняется только трактовка «сейчас» для будущих уведомлений и weekly-окна.
- **Лимиты Telegram:** длинные сообщения (>4096 символов) разбиваются. В weekly мы не ограничиваем длину ответов ученика, но при выводе истории — обрезаем до 120 символов с `…`.
- **Логирование действий ментора:** не критично для MVP, но желательно писать в `INFO` все изменения данных (кто, что, когда) — пригодится при разборе споров.

---

## 8. Вне скоупа MVP

Чтобы зафиксировать границы — следующее **не делаем** в этой итерации:

- Веб-админка / отдельный личный кабинет вне бота.
- История всех weekly для просмотра учеником.
- Приватные заметки ментора по ученику (поле `student_stage_progress.mentor_comment` есть в схеме, но не используется).
- Управление индивидуальными задачами через `student_tasks` (структура есть, но функциональности нет).
- Импорт прогресса из Excel (`progress_source='excel_import'` зарезервирован).
- Редактирование уже отправленного weekly.
- Прикрепление файлов / ссылок / скриншотов в weekly.
- Мульти-менторство (явная связь mentor↔student): в MVP считаем, что менторы видят всех учеников.
- Локализация (только русский).

---

## 9. Открытые вопросы / решения, требующие подтверждения

1. **Хостинг и деплой:** есть требования (свой VPS / контейнерный хостинг / Railway / др.)?
2. **ID общего чата учеников (`STUDENTS_CHAT_ID`):** ментор добавит бот в чат и пришлёт ID — или это автоматизировать?

3. `daily_progress.status`: в MVP используем только `active`. Подтвердить, что не нужно создавать записи `no_response` за пропущенные недели.
4. **Длина weekly-ответов:** ограничить (например, 2000 символов на поле) или оставить без ограничений?
5. **Добавить** `students.completed_at date` в схему (для отчёта «Получили оффер»)? Сейчас есть только `dropped_at`.
6. **Список TZ для регистрации (§5.1, шаг 5):** перечень в ТЗ — стартовый; нужно ли расширить (Европа, США, Азия за пределами СНГ)?

---

## 10. Чек-лист готовности MVP

- [ ] Миграции БД (дополнения из §4) применены.
- [ ] Первый ментор создан из env.
- [ ] Регистрация ученика по коду работает end-to-end.
- [ ] Weekly-кнопка появляется по понедельникам, FSM из 3 шагов сохраняет в БД.
- [ ] Кабинет ментора: поиск, карточка, перевод этапов, дедлайны, отчисление, оффер.
- [ ] CRUD направлений и этапов.
- [ ] Excel-экспорт «Общий прогресс».
- [ ] Уведомления: WELCOME, STAGE_MOVED, NEW_DEADLINES, WEEKLY_REMINDER (×3), DEADLINE_APPROACHING, DEADLINE_OVERDUE, STUDENT_NO_RESPONSE, STUDENT_OFFER. Дедупликация работает.
- [ ] Кабинет ученика: прогресс, дедлайны.
- [ ] Логи + базовая обработка ошибок Telegram API.
- [ ] README с инструкцией по запуску.